

수기 데이터에서 관계형 데이터베이스로의 전환

도서 대여 장부 사례로 배우는 정규화와 코드 테이블 설계

관련 교안 : 5. DB모델링 (모델링 개념·ERD·정규화), 6. DDL (제약조건·CREATE TABLE)

비고 : 등장하는 인물·연락처는 교육용으로 구성한 가상의 예시입니다.

DB 모델링 순서



#	단계	해야 할 일
1	문제파악 (1절)	수기 장부 문제점 분석 (데이터 중복·불일치, 원자값 위반, 식별 불가, 상태값 관리 미흡)
2	요구사항정의 (2.1~2.3절)	사장님 인터뷰 기반 요구사항 정의서 작성 (REQ-1~REQ-6 도출)
3	요구사항분석 (2.4절)	요구사항 문장에서 개체·속성·관계 도출
4	개념적설계 (3절)	엔티티 분리 (회원-도서 M:N을 대여 연결 엔티티로 해소)
5	논리적설계 (정규화·4절)	정규화(1~3차) 적용 및 코드 테이블 도입
6	물리적설계 (5~7절)	최종 ERD 확정 및 DDL 작성

1. 수기 장부 데이터 현황 및 문제점

작은 동네 도서 대여점이 지금까지 아래와 같은 종이(또는 엑셀 한 장) 장부에 대여 기록을 적어왔다고 합니다.

원본 수기 장부 샘플

대여일	회원명	연락처	주소	도서명	대여상태
2026-08-01	하람	010-1111-2222	서울시 마포구	자바의 정석 (상)	대여중
2026-08-01	하람	010-1111-2222	서울시 마포구	자바의 정석 (하)	반납완료
2026-08-02	은결	010-3333-4444	서울시 서대문구	SQL 기초, 자바의 정석 (상)	대여중
2026-08-03	하람	010-9999-8888	서울시 마포구	자바의 정석 (상)	대여중

주요 문제점 분석

- 데이터 중복 및 불일치:** 위 표에서 붉게 표시된 것처럼, 같은 '하람'인데 8/1 행에는 연락처가 010-1111-2222, 8/3 행에는 010-9999-8888 로 서로 다릅니다. 회원 정보가 대여 기록마다 반복 저장되다 보니, 번호가 바뀌었을 때 이미 적어둔 과거 행까지 전부 찾아 고치지 않으면 이런 불일치가 생깁니다.
- 원자값 위반:** 은결의 8/2일 행은 '도서명' 한 칸에 SQL 기초, 자바의 정석 (상) 처럼 두 권이 함께 적혀 있습니다. 한 칸(컬럼)에는 더 이상 쪼갤 수 없는 값 하나 — **원자값(Atomic Value)** — 만 담아야 하는데, 이 칸은 두 값을 한꺼번에 담고 있어 그 원칙을 어기고 있습니다. 이 상태로는 "은결이 SQL 기초를 반납했는지"만 따로 조회하거나 수정할 수 없습니다. (이 문제를 정식으로 다루는 정규화 개념은 4절에서 설명합니다.)
- 동명이인 및 동일 도서 식별 불가:** 회원명·도서명은 텍스트일 뿐 고유 식별자가 아니므로, 이름이 같은 다른 회원이나 같은 제목의 다른 책(예: 개정판)이 들어오면 서로 구별할 방법이 없습니다.
- 상태 값 관리 미흡:** '대여중'/'반납완료'를 사람이 손으로 적다 보면 '대여중 '(공백)', '대여중임' 같은 오타자가 섞여 들어가고, 이러면 "대여중인 책 목록"을 정확히 조회할 수 없게 됩니다.

이상현상(Anomaly) 관점에서 다시 보기

앞서 확인한 문제점들은 정규화 이론에서 **삽입이상·수정이상·삭제이상** 세 가지로 분류됩니다. 위 장부 예시로 하나씩 짚어보겠습니다.

- 삽입이상 (Insertion Anomaly):** 아직 책을 한 권도 빌리지 않은 신규 회원을 등록하려면, 대여일·도서명·대여상태 같은 칸을 어쩔 수 없이 비워두거나 임의의 값으로 채워야 합니다. 이 장부는 "대여 기록 한 줄 = 회원 정보 한 벌"이 묶여 있는 구조라서, 대여 사실이 없으면 회원 정보만 따로 저장할 방법이 없습니다.
- 수정이상 (Update Anomaly):** 문제점 1에서 본 하람의 연락처 불일치가 바로 이 경우입니다. 같은 회원 정보가 대여 기록마다 중복 저장되어 있어, 연락처가 바뀌면 그 회원이 등장하는 모든 행을 찾아 전부 고쳐야 합니다. 한 행이라도 놓치면 같은 회원인데 서로 다른 값이 공존하는 불일치가 생깁니다.

7. **삭제이상 (Deletion Anomaly)**: 은결이 8/2일에 빌린 'SQL 기초' 대여 행을 반납 처리 후 지운다고 가정하면, 그 행에만 존재하던 'SQL 기초'라는 도서 정보 자체가 장부에서 통째로 사라집니다. "대여가 끝났다"는 사실 하나를 지우려다 "그런 책이 있었다"는 사실까지 함께 잃어버리는 문제입니다.

2. 전산화 요구사항 정의

2.1 왜 전산화를 시도했는가

매장 회원이 늘고 하루 대여 건수가 많아지면서, 사장님은 1절에서 확인한 문제들이 "가끔 생기는 실수"가 아니라 **매일 반복되는 업무 장애**라는 것을 체감하기 시작했습니다. 특히 회원 연락처가 바뀔 때마다 예전 장부 페이지를 전부 뒤져서 고쳐야 하는 일이 잦아지자, 개발자에게 "지금 장부 관리가 왜 불편한지"를 정리해 전달하며 전산화를 의뢰했습니다. 이 대화 내용이 곧 요구사항 정의의 출발점이 됩니다.

2.2 요구사항 정의서 도출 과정

개발자는 1절의 문제점을 사장님과 다시 하나씩 확인하며, "시스템이 있다면 이걸 어떻게 보장해줘야 하는가"를 문장으로 바꿔봅니다. 문제점 하나가 요구사항 하나로 바로 이어지기도 합니다.

1절에서 확인한 문제점	사장님과 함께 정리한 요구사항
① 데이터 중복 및 불일치(연락처)	REQ-1: 회원 정보는 한 번만 등록하고, 이후 모든 대여 기록에서 재사용해야 한다. 연락처가 바뀌면 한 곳만 고쳐도 전체에 반영되어야 한다.
② 원자값 위반(도서명 여러 개)	REQ-2: 하나의 대여 기록에는 반드시 도서 한 권만 연결되어야 한다. 여러 권을 빌리면 대여 기록도 각각 따로 남아야 한다.
③ 동명이인 및 동일 도서 식별 불가	REQ-3: 회원과 도서는 시스템이 자동으로 부여하는 고유 번호로 구분할 수 있어야 한다.
④ 상태 값 관리 미흡	REQ-4: 대여 상태는 미리 정해진 값 중에서만 선택할 수 있어야 하고, 관리자가 상태 종류를 추가·수정할 수 있어야 한다.

대화를 이어가는 과정에서, 사장님은 지금 장부에 이미 있지만 "문제"로는 미처 인식하지 못했던 업무 규칙도 추가로 짚어주었습니다.

- "한 회원이 여러 책을 동시에 빌리기도 하고, 인기 있는 책은 여러 회원이 돌아가며 빌려요" → **REQ-5**
- "언제 빌려서 언제 반납했는지는 꼭 남아야 해요. 아직 안 돌려줬으면 반납일은 비워두면 되고요" → **REQ-6**

2.3 요구사항 정의서

지금까지의 대화를 표로 정리하면, 이후 모델링의 기준이 될 최종 요구사항 정의서가 완성됩니다.

번호	요구사항	근거
REQ-1	회원 정보(이름, 연락처, 주소)는 한 번만 등록하고, 모든 대여 기록에서 재사용한다.	문제점 ①
REQ-2	하나의 대여 기록에는 도서 한 권만 연결한다.	문제점 ②
REQ-3	회원과 도서는 시스템이 자동 부여하는 고유 번호로 구분한다.	문제점 ③
REQ-4	대여 상태는 미리 정의된 값 중에서만 선택하며, 관리자가 상태 종류를 관리할 수 있어야 한다.	문제점 ④
REQ-5	한 회원은 여러 도서를, 한 도서는 여러 회원에게(시간 차를 두고) 대여될 수 있어야 한다.	업무 특성 확인
REQ-6	대여일과 반납일을 기록하며, 반납 전에는 반납일이 비어 있어야 한다.	업무 특성 확인

2.4 요구사항 분석 — 개체 · 속성 · 관계 도출

요구사항 문장에서 명사는 개체(Entity) 후보, 명사가 가진 특징은 속성(Attribute), 동사는 관계(Relationship) 후보가 됩니다.

요구사항	추출된 개체/속성	추출된 관계
REQ-1	회원(MEMBER): 이름, 연락처, 주소	-
REQ-2, REQ-5	도서(BOOK), 대여(RENTAL)	회원 1—N 대여, 도서 1—N 대여 (회원—도서의 M:N을 대여로 해소, 3절에서 자세히 설명)
REQ-3	회원ID(PK), 도서ID(PK)	-
REQ-4	대여상태코드(RENTAL_STATUS_CODE): 코드, 상태명	대여상태코드 1—N 대여
REQ-6	대여(RENTAL): 대여일, 반납일	-

주의 포인트: REQ-5는 "회원도 여러 책, 책도 여러 회원"이라고 말하고 있으므로 문자 그대로 보면 M:N 관계입니다. M:N은 물리적으로 구현할 수 없으므로, REQ-6이 요구하는 "대여일/반납일"이라는 관계 자체의 속성을 담을 연결 엔티티(대여)를 만들어 1:N + 1:N으로 풀어냅니다. 자세한 이유는 바로 다음 절에서 확인합니다.

3. 개념적 모델링 (엔티티 분리)

3.1 첫 번째 시도 — 회원과 도서를 M:N으로 직접 연결

REQ-5("한 회원은 여러 도서를, 한 도서는 여러 회원에게 대여될 수 있어야 한다")만 놓고 보면, 가장 먼저 떠오르는 설계는 회원과 도서를 곧바로 연결하는 것입니다. 한 회원이 여러 책을 빌릴 수 있고, 한 책도 시간 차를 두고 여러 회원에게 빌려질 수 있으므로 이 관계는 **M:N**입니다.



⚠ 테이블 두 개만으로는 구현할 수 없는 관계

이 M:N 관계를 실제 테이블로 옮기려면, 회원 테이블(또는 도서 테이블) 한쪽에 상대편 번호를 "몰아서" 적는 방법밖에 없습니다. 예를 들어 회원 테이블에 지금 빌린 도서번호를 나열해보면 다음과 같습니다.

member_id	name	phone	borrowed_book_ids
1	하람	010-9999-8888	1, 2
2	은결	010-3333-4444	3, 1

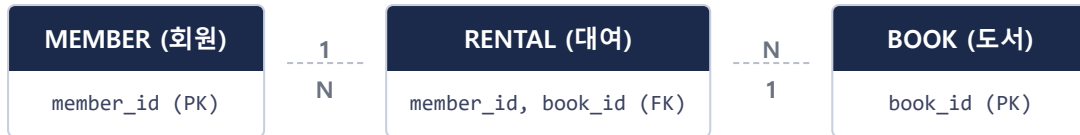
3.2 왜 이 방식이 안 되는가

- 원자값 위반 재발:** `borrowed_book_ids` 컬럼에 여러 값이 한꺼번에 들어가 있습니다. 1절에서 이미 해결했던 "도서명 여러 권이 한 칸에" 문제(원자값 위반)가 회원—도서 관계에서 그대로 되풀이된 것입니다. "하람이 2번 책을 반납했는지"만 따로 고치려면 콤마로 이어붙인 문자열을 직접 잘라내 수정해야 합니다.
- 관계 자체의 정보를 저장할 곳이 없음:** REQ-6은 "대여일과 반납일을 기록해야 한다"고 요구하는데, 이 값은 회원의 속성도 도서의 속성도 아니라 **"이 회원이 이 책을 빌린 사건" 하나에 속하는 값**입니다. 대여일 컬럼을 회원 테이블에 하나 더 추가해도, 책을 두 권 빌리면 대여일도 두 개 필요해져 똑같은 원자값 위반이 반복됩니다.
- 같은 책을 다시 빌린 이력을 구별할 수 없음:** 하람이 1번 책을 반납했다가 나중에 또 빌리면, 콤마 목록에는 "1"이 두 번 들어가야 하는데 어떤 게 이번 대여이고 어떤 게 지난번 대여였는지 구별할 방법이 없습니다.

결론적으로, 관계형 데이터베이스의 테이블은 한 행이 가지는 컬럼 개수가 고정되어 있어서 **M:N 관계를 테이블 두 개만으로는 물리적으로 표현할 수 없습니다** (`2_ERD와정규화.md` 에서 배운 원칙과 동일합니다).

3.3 해결책 — 대여(RENTAL) 연결 테이블 도입

"이 회원이 이 책을 빌린 사건" 자체를 별도의 엔티티로 승격시키면 세 가지 문제가 한 번에 풀립니다. **대여 (RENTAL)**라는 새 엔티티를 만들어, 회원—대여(1:N), 도서—대여(1:N) 두 개의 1:N 관계로 풀어냅니다.



대여일·반납일처럼 "사건 자체에 속하는 값"은 이제 RENTAL의 속성으로 자연스럽게 들어갈 자리가 생기고, 같은 책을 여러 번 빌려도 RENTAL 행이 매번 새로 추가되므로 이력도 전부 구별됩니다. 최종적으로 정리하는 엔티티는 다음 세 가지입니다.

엔티티	역할
회원 (MEMBER)	회원 정보를 단 한 번만 저장하여 중복을 제거한다.
도서 (BOOK)	도서 개별 정보를 고유 ID로 관리해 동명 도서를 구별한다.
대여 (RENTAL)	어떤 회원이 어떤 도서를 언제 대여하고 반납했는지 매핑한다.

4. 논리적 모델링 — 정규화 및 코드 테이블 도입

정규화(Normalization)란 테이블을 중복과 이상현상(삽입·수정·삭제 이상) 없이 저장되도록 단계적으로 분해해 가는 과정입니다. 이 과정에서 지켜야 할 규칙의 단계를 **정규형(Normal Form)**이라 부르며, 어느 단계까지 규칙을 만족했는지에 따라 제1정규형(1NF), 제2정규형(2NF), 제3정규형(3NF)... 으로 구분합니다.

제1정규형(1NF)의 조건은 "모든 컬럼이 **원자값(Atomic Value, 더 이상 쪼갤 수 없는 단일 값)**만 가져야 한다"는 것입니다. 1절에서 확인한 '원자값 위반' 문제(도서명 한 칸에 두 권이 함께 적힌 것)가 바로 이 제1정규형을 위반한 사례입니다.

① 제1정규형 (1NF) — 원자값 위반 해소

한 컬럼에는 하나의 값만 담아야 하므로, 은결의 8/2 행처럼 여러 도서가 한 칸에 적힌 경우 대여 건별로 행을 분리합니다.

대여일	회원명	연락처	주소	도서명	대여상태
2026-08-01	하람	010-1111-2222	서울시 마포구	자바의 정석 (상)	대여중
2026-08-01	하람	010-1111-2222	서울시 마포구	자바의 정석 (하)	반납완료
2026-08-02	은결	010-3333-4444	서울시 서대문구	SQL 기초	대여중
2026-08-02	은결	010-3333-4444	서울시 서대문구	자바의 정석 (상)	대여중
2026-08-03	하람	010-9999-8888	서울시 마포구	자바의 정석 (상)	대여중

행이 4개에서 5개로 늘었지만, 이제 "SQL 기초 대여 건"과 "자바의 정석 (상) 대여 건"을 각각 독립적으로 조회·수정할 수 있습니다. 다만 하람의 연락처 불일치(문제점 1)는 아직 그대로 남아 있습니다 — 이는 1NF가 아니라 다음 단계(엔티티 분리)에서 해결됩니다.

② 제2정규형 (2NF) — 완전 함수 종속과 회원·도서 정보 분리

제2정규형의 조건은 "1NF를 만족하면서, 기본키가 아닌 모든 속성이 기본키 전체에 **완전 함수 종속**해야 한다"는 것입니다. 기본키가 여러 컬럼으로 이루어진 경우, 그 일부에만 종속되는 속성이 있으면 안 됩니다 — 이를 **부분 함수 종속**이라 하고, 2NF는 이 부분 함수 종속을 제거하는 단계입니다.

바로 위 1NF 결과 표를 보면, 한 행을 구별하는 사실상의 키는 "회원명 + 도서명 + 대여일" 조합입니다. 그런데 연락처·주소는 이 키 전체가 아니라 '**회원명**' 하나에만 종속됩니다. 하람이 어떤 책을 몇 번 빌리든 연락처는 항상 같은 값이어야 하므로, 이 값은 사실 '회원명'만으로 이미 결정되는 값이지 "회원명+도서명+대여일" 조합 전체가 있어야 정해지는 값이 아닙니다. 저자(author)도 마찬가지로 '도서명'에만 종속되고, 대여일·대여상태와는 무관합니다. 이렇게 키의 일부에만 종속되는 속성이 섞여 있는 것이 부분 함수 종속이며, 2NF 위반입니다.

이 부분 종속을 없애려면, 회원명에만 종속되는 속성(연락처, 주소)은 **MEMBER** 테이블로, 도서명에만 종속되는 속성(저자)은 **BOOK** 테이블로 각각 분리해 자신만의 기본키(member_id, book_id)에 완전히 종속시킵니다.

대여 기록마다 반복 저장되던 회원 정보가 회원당 단 한 행에만 존재하게 되므로, 연락처가 바뀌어도 그 한 행만 고치면 되어 문제점 1(데이터 불일치)이 근본적으로 해소됩니다.

③ 제3정규형 (3NF) — 이행적 종속 제거

제3정규형의 조건은 "2NF를 만족하면서, 기본키가 아닌 속성끼리 서로를 결정하는 관계(**이행적 종속**)가 없어야 한다"는 것입니다. 즉 `기본키 → A → B` 처럼, B가 기본키가 아니라 다른 비(非)키 속성 A를 거쳐서만 결정되는 경우가 있으면 안 됩니다.

예를 들어 만약 BOOK 테이블에 저자명뿐 아니라 **저자의 소속 출판사**까지 함께 저장한다면 어떻게 될까요. 소속 출판사는 `book_id`가 아니라 `author` (저자명)라는 값에 의해서만 결정됩니다. 즉 `book_id → author → 소속 출판사` 처럼 `author`를 한 단계 거쳐야 정해지는 값이므로 이행적 종속에 해당하고, 이 경우 저자 정보를 AUTHOR라는 별도 테이블로 다시 분리해야 3NF를 만족합니다.

이번 실습에서는 MEMBER의 비PK 속성(name, phone, address)과 BOOK의 비PK 속성(title, author)이 모두 각 테이블의 PK(member_id, book_id)에 직접 종속될 뿐, 다른 비PK 속성을 거쳐 결정되는 값이 없습니다. 따라서 2NF까지만 적용해도 이행적 종속이 애초에 존재하지 않아 자연스럽게 3NF까지 만족합니다.

④ 코드 테이블 분리 — 상태 값 표준화

'대여중', '반납완료', '연체' 같은 상태 값을 자유 텍스트로 두지 않고, 아래처럼 별도의 코드 테이블로 관리하면 오타자가 원천적으로 불가능해지고, 나중에 새 상태(예: '분실')를 추가하기도 쉬워집니다.

코드 (status_code)	상태명 (status_name)
RENT	대여중
RETURN	반납완료
OVERDUE	연체중

5. 물리적 모델링 — 최종 ERD 스키마 설계

지금까지 개념적 모델링(3절, 엔티티와 관계 도출)과 논리적 모델링(4절, 정규화·코드 테이블)을 거쳤습니다. 이제 실제 MySQL에 저장할 수 있도록 컬럼마다 구체적인 데이터 타입과 제약조건을 확정하는 **물리적 모델링** 단계입니다. 이어지는 6절(데이터로 확인)과 7절(DDL 실행)까지가 모두 이 물리적 모델링에 해당합니다.

[1] 회원 테이블 (MEMBER)

컬럼명 (ENG)	컬럼명 (KOR)	제약조건	비고
member_id	회원ID	PK, AUTO_INCREMENT	고유 식별자
name	회원명	NOT NULL	
phone	연락처	NOT NULL	최신 정보 유지
address	주소	NULL	

[2] 도서 테이블 (BOOK)

컬럼명 (ENG)	컬럼명 (KOR)	제약조건	비고
book_id	도서ID	PK, AUTO_INCREMENT	동명 도서 구별
title	도서명	NOT NULL	
author	저자	NULL	

[3] 대여상태 코드 테이블 (RENTAL_STATUS_CODE)

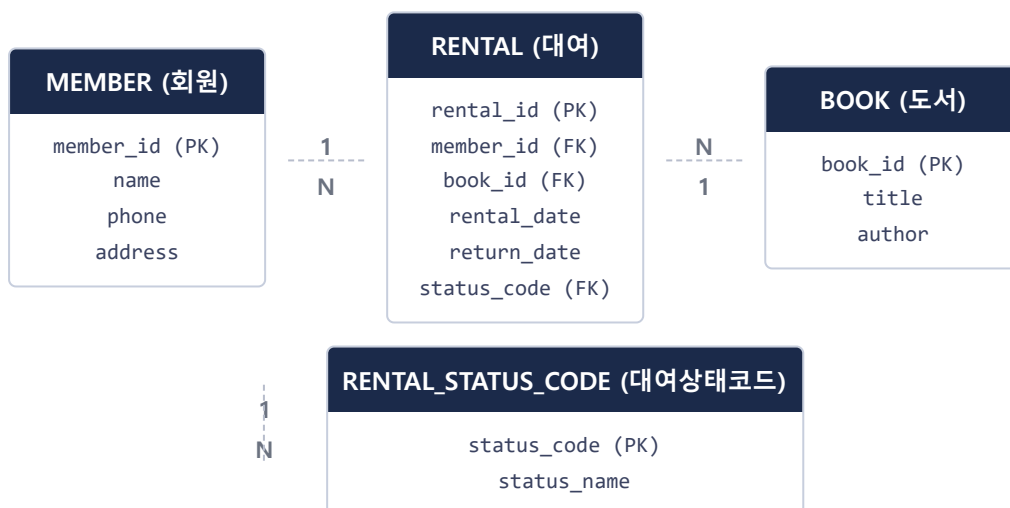
코드 (status_code)	상태명 (status_name)
RENT	대여중
RETURN	반납완료
OVERDUE	연체중

[4] 대여 테이블 (RENTAL)

컬럼명 (ENG)	컬럼명 (KOR)	제약조건	비고
rental_id	대여ID	PK, AUTO_INCREMENT	
member_id	회원ID	FK (MEMBER.member_id)	N:1 관계
book_id	도서ID	FK (BOOK.book_id)	N:1 관계
rental_date	대여일	NOT NULL	
return_date	반납일	NULL	반납 전이면 NULL
status_code	대여상태코드	FK (RENTAL_STATUS_CODE.status_code)	상태 표준화

날짜 컬럼 타입: 원본 장부에는 "몇 시에 빌렸는지"가 아니라 "며칠에 빌렸는지"만 기록되어 있으므로, 0. 환경구성/01_데이터타입.md 기준으로 rental_date / return_date 는 시각까지 필요한 DATETIME 이 아니라 DATE 타입으로 설계합니다. 만약 실제 대여/반납 시각까지 기록해야 한다면 DATETIME 으로 바꿉니다.

ER 다이어그램



점선(선택 참여)은 예를 들어 회원(MEMBER)은 아직 한 번도 책을 빌리지 않았어도 존재할 수 있지만(회원 1 — 대여 0..N), 대여(RENTAL) 한 건은 반드시 회원 1명, 도서 1권, 상태코드 1개를 가져야 한다는 의미입니다.

6. 물리적 모델링 — 정규화된 데이터로 다시 채워보기

3~4절에서 설계한 4개 테이블에 1절의 수기 장부 데이터를 옮겨 담으면 다음과 같습니다.

MEMBER

member_id	name	phone	address
1	하람	010-9999-8888	서울시 마포구
2	은결	010-3333-4444	서울시 서대문구

하람의 연락처는 가장 최근 값(010-9999-8888)으로 **단 한 행에만** 저장됩니다. 더 이상 "어느 행을 고쳐야 하는지" 고민할 필요가 없습니다.

BOOK

book_id	title	author
1	자바의 정석 (상)	남궁성
2	자바의 정석 (하)	남궁성
3	SQL 기초	NULL

RENTAL_STATUS_CODE

status_code	status_name
RENT	대여중
RETURN	반납완료
OVERDUE	연체중

RENTAL

rental_id	member_id	book_id	rental_date	return_date	status_code
1	1 (하람)	1 (자바의 정석 (상))	2026-08-01	NULL	RENT
2	1 (하람)	2 (자바의 정석 (하))	2026-08-01	2026-08-06	RETURN
3	2 (은결)	3 (SQL 기초)	2026-08-02	NULL	RENT
4	2 (은결)	1 (자바의 정석 (상))	2026-08-02	NULL	RENT
5	1 (하람)	1 (자바의 정석 (상))	2026-08-03	NULL	RENT

rental_id 1, 4, 5는 모두 같은 책(book_id=1, 자바의 정석 (상))을 가리킵니다. 도서 정보를 매번 다시 입력하지 않고 **도서ID 하나로 재사용**할 수 있다는 것이 도서를 별도 테이블로 분리한 효과입니다.

7. 물리적 모델링 — CREATE TABLE DDL (MySQL)

6. DDL/1_DDL.md 에서 배운 PRIMARY KEY, FOREIGN KEY, NOT NULL 제약조건을 그대로 적용한 실행 가능한 스크립트입니다.

7.1 테이블 생성

```
CREATE TABLE MEMBER (  
    MEMBER_ID INT          AUTO_INCREMENT PRIMARY KEY,  
    NAME      VARCHAR(20)  NOT NULL,  
    PHONE     VARCHAR(20)  NOT NULL,  
    ADDRESS   VARCHAR(100)  
);  
  
CREATE TABLE BOOK (  
    BOOK_ID INT          AUTO_INCREMENT PRIMARY KEY,  
    TITLE   VARCHAR(100) NOT NULL,  
    AUTHOR  VARCHAR(50)  
);  
  
CREATE TABLE RENTAL_STATUS_CODE (  
    STATUS_CODE VARCHAR(10) PRIMARY KEY,  
    STATUS_NAME VARCHAR(20) NOT NULL  
);  
  
CREATE TABLE RENTAL (  
    RENTAL_ID    INT          AUTO_INCREMENT PRIMARY KEY,  
    MEMBER_ID    INT          NOT NULL,  
    BOOK_ID      INT          NOT NULL,  
    RENTAL_DATE  DATE         NOT NULL,  
    RETURN_DATE  DATE,  
    STATUS_CODE  VARCHAR(10)  NOT NULL,  
    CONSTRAINT FK_RENTAL_MEMBER FOREIGN KEY (MEMBER_ID) REFERENCES MEMBER (MEMBER_ID),  
    CONSTRAINT FK_RENTAL_BOOK   FOREIGN KEY (BOOK_ID)   REFERENCES BOOK (BOOK_ID),  
    CONSTRAINT FK_RENTAL_STATUS FOREIGN KEY (STATUS_CODE) REFERENCES RENTAL_STATUS_CODE (STATUS_CODE)  
);
```

7.2 샘플 데이터 입력

```
-- 코드 테이블은 데이터 종류가 고정적이므로 먼저 채워준다
INSERT INTO RENTAL_STATUS_CODE VALUES ('RENT', '대여중');
INSERT INTO RENTAL_STATUS_CODE VALUES ('RETURN', '반납완료');
INSERT INTO RENTAL_STATUS_CODE VALUES ('OVERDUE', '연체중');

-- 6절의 정규화 결과 데이터
INSERT INTO MEMBER (NAME, PHONE, ADDRESS) VALUES ('하람', '010-9999-8888', '서울시 마포구');
INSERT INTO MEMBER (NAME, PHONE, ADDRESS) VALUES ('은결', '010-3333-4444', '서울시 서대문구');

INSERT INTO BOOK (TITLE, AUTHOR) VALUES ('자바의 정석 (상)', '남궁성');
INSERT INTO BOOK (TITLE, AUTHOR) VALUES ('자바의 정석 (하)', '남궁성');
INSERT INTO BOOK (TITLE, AUTHOR) VALUES ('SQL 기초', NULL);

INSERT INTO RENTAL (MEMBER_ID, BOOK_ID, RENTAL_DATE, RETURN_DATE, STATUS_CODE) VALUES
(1, 1, '2026-08-01', NULL, 'RENT'),
(1, 2, '2026-08-01', '2026-08-06', 'RETURN'),
(2, 3, '2026-08-02', NULL, 'RENT'),
(2, 1, '2026-08-02', NULL, 'RENT'),
(1, 1, '2026-08-03', NULL, 'RENT');
```

생성 순서에 주의: 외래키가 참조하는 테이블이 먼저 있어야 하므로 `MEMBER` / `BOOK` / `RENTAL_STATUS_CODE` → `RENTAL` 순서로 실행합니다. 마찬가지로 데이터를 지울 때는 `RENTAL` 을 먼저 지워야 6. DDL/1_DDL.md 에서 다룬 참조 무결성 오류를 피할 수 있습니다.

8. 문제점 ↔ 요구사항 ↔ 해결책 매핑 체크리스트

2절에서 뽑아낸 요구사항이 실제로 최종 설계에 빠짐없이 반영되었는지 마지막으로 확인합니다.

1절 문제점	요구사항	이 설계에서의 해결책	확인
데이터 중복 및 불일치 (연락처)	REQ-1	MEMBER 테이블에 회원당 1행만 저장, RENTAL은 member_id(FK)로만 참조	✓
원자값 위반 (도서명 여러개)	REQ-2	1NF로 대여 건별 1행 분리 + RENTAL이 book_id(FK) 하나만 가짐	✓
동명이인 / 동일 도서 식별 불가	REQ-3	MEMBER.member_id, BOOK.book_id를 AUTO_INCREMENT PK로 부여	✓
상태 값 이탈자 위험	REQ-4	RENTAL_STATUS_CODE 코드 테이블 분리 + RENTAL.status_code FK 제약	✓
(추가 확인된 업무 규칙)	REQ-5	MEMBER 1—N RENTAL, BOOK 1—N RENTAL로 회원—도서 M:N을 해소	✓
(추가 확인된 업무 규칙)	REQ-6	RENTAL.rental_date NOT NULL, RENTAL.return_date NULL 허용	✓

이 문서는 교안/SQL/5. DB모델링, 6. DDL 챕터 학습 직후 "수기 장부 하나를 실제로 끝까지 정규화해보는" 실습/시연 자료로 사용할 수 있습니다.